

Методический материал	3
Ручное тестирование для .NET и Unity-разработчика	3
2. Базовая терминология.....	3
3. SUT — тестируемая система.....	4
4. Требования.....	5
5. Test Case и Test Scenario	6
6. Checklist.....	7
7. Bug, Defect, Error, Failure.....	7
8. Bug Report.....	8
9. Severity и Priority (важность и приоритет).....	9
10. Smoke Testing.....	9
11. Sanity Testing.....	10
12. Regression Testing.....	10
13. Retest.....	11
14. Exploratory Testing.....	11
15. Ad-hoc Testing.....	12
16. Техники тест-дизайна	12
17. Equivalence Partitioning (разделение по эквивалентностям)	12
18. Boundary Value Analysis.....	13
19. Pairwise Testing (парное тестирование).....	13
20. Decision Table (таблица соответствий)	14
21. State Transition Testing.....	15
22. Positive и Negative Testing.....	15
23. Уровни тестирования	16
24. Backend/API Testing для .NET.....	17
25. API Payload Testing.....	18
26. Idempotency	18

27. Rate Limiting.....	18
28. Client-Server Testing.....	19
29. Unity: Game-Specific Testing.....	19
30. Memory Testing.....	20
31. Unity Lifecycle Testing.....	20
32. Physics / Collision Testing.....	21
33. Flaky Bug.....	21
34. Environment.....	22
35. Build Type.....	23
36. Работа с логами.....	23
37. Network Tracing.....	24
38. Жизненный цикл бага.....	25
39. Практическая методика тестирования новой функции.....	26
40. Методика тестирования Unity-фичи.....	27
41. Методика тестирования .NET API.....	28
42. Что должен проверять разработчик перед передачей задачи QA.....	29
43. Что особенно важно для Unity-разработчика.....	30
44. Что особенно важно для .NET-разработчика.....	31
45. Ключевые вопросы разработчика при тестировании.....	32
46. Минимальный набор терминов для разработчика.....	32
47. Итоговая модель мышления.....	34

Методический материал

Ручное тестирование для .NET и Unity-разработчика

1. Цель материала

Ручное тестирование — это не только работа QA-инженера. Разработчик должен уметь самостоятельно проверить реализованную функциональность до передачи задачи на code review, QA или заказчику.

Для разработчика особенно важно научиться отвечать на вопросы:

Что должна делать система?

Что произойдет при корректных данных?

Что произойдет при некорректных данных?

Что произойдет на границе допустимых значений?

Что произойдет при неожиданном действии пользователя?

Не сломал ли новый код существующую функциональность?

2. Базовая терминология

2.1. Software Testing

Software Testing — процесс проверки программного обеспечения для обнаружения дефектов и оценки соответствия системы установленным требованиям.

Пример:

Проверяем, что после покупки машины она появляется в гараже, деньги списываются, а состояние сохраняется.

2.2. Manual Testing

Manual Testing — тестирование, выполняемое человеком без автоматического выполнения тестовых сценариев.

Например:

Запустить игру

→ Открыть гараж

→ Выбрать машину

→ Нажать «Купить»

→ Проверить баланс

- Перезапустить игру
- Проверить сохранение

2.3. Automated Testing

Automated Testing — тестирование, при котором проверки выполняются программой или автоматизированным тестовым фреймворком.

Для .NET это, например:

- NUnit
- xUnit
- MSTest

Для Unity также можно использовать Unity Test Framework.

2.4. QA

QA (Quality Assurance) — обеспечение качества программного продукта посредством процессов, практик и контроля качества на всех этапах разработки.

QA — это шире, чем просто поиск багов.

2.5. QC

QC (Quality Control) — контроль качества непосредственно создаваемого продукта, в частности путем проведения тестирования и выявления дефектов.

Упрощенно:

QA — как организовать качество. QC — как проверить качество результата.

3. SUT — тестируемая система

SUT (System Under Test) — система, которая в данный момент тестируется.

Для .NET-проекта это может быть:

ASP.NET Core API

+

Database

+
Frontend
Для Unity:
Unity Build
+
Backend API
+
Save System
+
Ads
+
IAP

Важно понимать, что SUT зависит от уровня тестирования.

Например, при тестировании REST API SUT может быть только API.

При E2E-тестировании SUT — практически вся система.

4. Требования

Requirement — требование к системе или функциональности.

Пример:

Игрок может приобрести автомобиль, если у него достаточно монет.

Functional Requirement — требование, описывающее, что система должна делать.

Пример:

При нажатии кнопки «Купить» машина должна перейти в состояние Purchased.

Non-Functional Requirement (NFR) — требование к характеристикам системы.

Например:

- FPS не ниже 30;
- API отвечает менее чем за 500 мс;
- приложение работает на iOS;
- система поддерживает 1000 одновременных пользователей.

Acceptance Criteria — условия, которым должна соответствовать реализованная функция, чтобы считаться принятой.

Например:

Given: у игрока 1500 монет

When: игрок покупает машину стоимостью 1000

Then:

- машина становится доступной;
- баланс становится 500;
- данные сохраняются.

5. Test Case и Test Scenario

Test Scenario — высокоуровневое описание того, что необходимо проверить.

Например:

Проверить покупку автомобиля.

Test Case — конкретный тест с предусловиями, входными данными, шагами и ожидаемым результатом.

Пример:

Preconditions:

Игрок имеет 1500 монет.

Steps:

1. Открыть гараж.
2. Выбрать машину стоимостью 1000.
3. Нажать «Купить».

Expected Result:

Машина куплена.

Баланс = 500.

Expected Result — ожидаемый результат выполнения теста.

Actual Result — фактический результат выполнения теста.

Если:

Expected: баланс = 500

Actual: баланс = 0

мы получили дефект.

6. Checklist

Checklist — список проверок без подробного описания всех шагов.

Например:

Авторизация

- ☐ Валидный логин
- ☐ Неверный пароль
- ☐ Пустой логин
- ☐ Пустой пароль
- ☐ Очень длинный пароль
- ☐ Повторная авторизация
- ☐ Потеря соединения

Checklist удобен для быстрого Smoke / Regression Testing.

7. Bug, Defect, Error, Failure

Эти термины желательно различать.

Error — ошибка человека, например разработчика.

`if (value > 100)`

вместо:

`if (value >= 100)`

Defect — дефект, присутствующий в программном обеспечении вследствие ошибки.

Failure — проявление дефекта во время выполнения программы.

Например:

Error

↓

Неправильное условие

↓

Defect

↓

Неверная логика

↓

Failure

↓

Программа не принимает значение 100

В повседневной работе все эти понятия часто называют **Bug**.

8. Bug Report

Bug Report — описание обнаруженного дефекта, позволяющее разработчику воспроизвести и исправить проблему.

Хороший Bug Report содержит:

- Summary (описание)
- Preconditions (вводные)
- Steps to Reproduce (шаги для воссоздания)
- Actual Result (фактический результат)
- Expected Result (ожидаемый результат)
- Environment (окружение/среда)
- Severity (важность)
- Priority (приоритет)
- Attachments (вложения)
- Logs (логи)
- Video / Screenshot

Пример

Summary:

Игра падает при покупке Heavy Car.

Preconditions:

На балансе 5000 монет.

Steps:

1. Открыть Garage.
2. Выбрать Heavy Car.
3. Нажать Buy.

Expected:

Машина покупается, баланс уменьшается на стоимость машины.

Actual:

Приложение закрывается.

Attachments:

Video + Player.log.

9. Severity и Priority (важность и приоритет)

Это одна из самых важных тем для разработчика.

Severity — серьезность дефекта с технической/функциональной точки зрения.

Условная шкала:

Уровень	Значение
S1 Critical	Crash, потеря данных, блокировка системы
S2 Major	Критически важная функция не работает
S3 Minor	Функциональность работает частично
S4 Trivial	UI, текст, косметические проблемы

Priority — срочность исправления дефекта с точки зрения бизнеса или релиза.

Например:

Severity = S4

Priority = P1

Ошибка в названии продукта может быть технически незначительной, но если она находится на главном экране перед релизом — бизнес может потребовать исправить ее немедленно.

Главное:

Severity отвечает на вопрос «**насколько плохо?**»

Priority отвечает на вопрос «**насколько срочно?**»

10. Smoke Testing

Smoke Testing — быстрая проверка наиболее критичных функций после получения новой сборки.

Для Unity:

Запуск
↓
Главное меню
↓
Гараж
↓
Выбор машины
↓
Старт гонки
↓
Финиш
↓
Награда
↓
Сохранение

Если игра не запускается — дальнейшее тестирование сборки может быть бессмысленным.

11. Sanity Testing

Sanity Testing — быстрая проверка конкретной области после небольшого изменения.

Например, исправили покупку автомобиля.

Не обязательно сразу проходить всю игру.

Сначала проверяем:

Магазин
→ Покупка
→ Баланс
→ Состояние машины

После этого уже выполняем Regression Testing.

12. Regression Testing

Regression Testing — проверка того, что существующая функциональность не была сломана внесенными изменениями.

Например, исправили систему сохранения.

Нужно проверить:

- загрузку;
- покупки;
- настройки;
- прогресс;
- валюту;
- выбранную машину;
- уровень игрока.

13. Retest

Retest — повторная проверка конкретного исправленного дефекта.

Например:

В версии 1.5 исправлен Crash при покупке машины.

QA повторяет именно тот сценарий, который ранее приводил к Crash.

Разница:

Retest:

Исправили ли этот конкретный баг?

Regression:

Не сломали ли изменения что-нибудь еще?

14. Exploratory Testing

Exploratory Testing — исследовательское тестирование, при котором тестировщик одновременно изучает систему, проектирует проверки и выполняет их.

Например, тестируя сохранение:

Что будет, если закрыть приложение именно в момент сохранения?

Для Unity особенно полезно проверять:

- быстрое нажатие кнопок;
- смену сцен;
- сворачивание приложения;
- потерю интернета;
- блокировку телефона;
- повторный запуск;

- быстрое переключение состояний.

15. Ad-hoc Testing

Ad-hoc Testing — неформальное тестирование без заранее подготовленного сценария.

Например:

«Попробую несколько раз быстро нажать кнопку покупки и посмотрю, не спишутся ли деньги дважды».

Это полезный подход, но он не заменяет систематические Test Cases и Checklists.

16. Техники тест-дизайна

Это один из наиболее важных разделов для разработчика.

Профессиональный тестировщик не просто «кликает все подряд».

Он выбирает тесты системно.

17. Equivalence Partitioning (разделение по эквивалентностям)

Equivalence Partitioning — разбиение входных данных на классы эквивалентности, внутри которых система должна вести себя одинаково.

Например:

Возраст: 18–60

Классы:

< 18	→ invalid
18–60	→ valid
> 60	→ invalid

Можно взять:

15
30
70

вместо проверки всех возможных значений.

18. Boundary Value Analysis

Boundary Value Analysis — анализ граничных значений.

Для:

18–60

проверяем:

17

18

19

59

60

61

Границы особенно важны, потому что здесь часто возникают ошибки:

>

>=

<

<=

а также:

- `IndexOutOfRangeException`;
- `ArgumentOutOfRangeException`;
- `OverflowException`.

19. Pairwise Testing (парное тестирование)

Pairwise Testing — техника сокращения количества комбинаций параметров.

Предположим, есть:

OS:

Windows

macOS

Linux

Language:

RU

EN

Resolution:
720p
1080p
1440p

GPU:
Low
Medium
High

Полный перебор даст множество комбинаций.

Pairwise Testing стремится обеспечить покрытие **пар значений параметров**, предполагая, что значительная часть дефектов возникает из-за взаимодействия двух факторов.

Важно:

Pairwise не гарантирует «95% багов». Это техника оптимизации количества тестов, а степень покрытия зависит от системы и набора параметров.

20. Decision Table (таблица соответствий)

Decision Table — таблица условий и соответствующих действий системы.

Например:

Есть деньги	Машина куплена	Результат	
Да	Нет	Купить	
Да	Да	Показать Purchased	Already
Нет	Нет	Показать Enough Coins	Not
Нет	Да	Already Purchased	

Особенно полезно для:

- бизнес-логики .NET;
- покупок;
- доступа пользователей;
- игровых механик.

21. State Transition Testing

State Transition Testing — проверка переходов системы между состояниями.

Для машины:

Locked
↓
Available
↓
Purchased
↓
Selected

Нужно проверить не только сами состояния, но и допустимые переходы.

Например:

Locked → Selected

может быть запрещен.

Для Unity это особенно полезная техника.

22. Positive и Negative Testing

Positive Testing

Проверяем корректные данные.

Balance = 5000
Car Price = 1000
→ Purchase

Negative Testing

Проверяем некорректные данные.

Balance = 500
Car Price = 1000
→ Purchase

Ожидаем:

Покупка невозможна.

Другой пример для API:

```
{  
  "coins": -5  
}
```

Если клиент не умеет корректно обработать такой ответ — это потенциальный дефект.

23. Уровни тестирования

Unit / Component Testing

Проверяется отдельный компонент.

Для .NET:

```
CalculatePrice()  
ValidateEmail()  
CalculateReward()
```

Для Unity:

```
CarController  
SaveService  
RewardCalculator
```

Обычно автоматизируется.

Integration Testing

Проверяется взаимодействие компонентов.

Например:

```
Unity  
↓  
.NET API  
↓  
Database
```

или:

```
.NET API  
↓  
PostgreSQL
```

Для ручного тестирования API удобно использовать:

- Swagger UI;
- Postman;

- другие HTTP-клиенты.

System Testing

Проверяется система целиком.

Например:

Launch Game

- Login
- Garage
- Buy Car
- Race
- Reward
- Save
- Restart
- Load

End-to-End (E2E) Testing — проверка полного пользовательского сценария через все необходимые компоненты системы.

Acceptance Testing — проверка соответствия продукта требованиям бизнеса или заказчика.

24. Backend/API Testing для .NET

При ручном тестировании API необходимо проверять не только 200 OK.

HTTP Status Codes

Минимально необходимо понимать:

Код	Значение
200	Успешный запрос
201	Ресурс создан
204	Успешно, без содержимого
400	Некорректный запрос
401	Не авторизован
403	Доступ запрещен
404	Ресурс не найден
409	Конфликт
429	Слишком много запросов
500	Ошибка сервера
502	Ошибка шлюза

25. API Payload Testing

Нужно проверять:

- структуру JSON;
- обязательные поля;
- типы данных;
- null;
- пустые значения;
- неизвестные поля;
- отрицательные значения;
- слишком большие значения.

Например:

```
{  
  "coins": -5  
}
```

Система должна корректно обработать некорректные данные.

26. Idempotency

Idempotency — свойство операции, при котором повторное выполнение одного и того же запроса не приводит к нежелательному повторному эффекту.

Очень важно для платежей и покупок.

Например:

POST Purchase
Idempotency-Key: ABC123

Повтор того же запроса не должен создать две покупки.

27. Rate Limiting

Rate Limiting — ограничение количества запросов от клиента за определенный промежуток времени.

Например:

100 requests / minute

Тестировщик может проверить:

- превышение лимита;
- HTTP 429;
- восстановление доступа после timeout window.

28. Client-Server Testing

Для Unity + .NET необходимо проверять не только сервер и клиент отдельно, но и их взаимодействие.

Например:

.NET:

coins = -5

↓

Unity:

UI отображает -5

Возникают вопросы:

- должен ли клиент доверять серверу?
- валидируется ли значение?
- может ли UI показать некорректное состояние?
- не возникает ли exception?

Это классический пример **Negative Testing**.

29. Unity: Game-Specific Testing

FPS / Performance

Недостаточно написать в Bug Report:

«Игра тормозит».

Нужно зафиксировать:

Device

Build

Scene

Average FPS

Minimum FPS

Duration

Steps

Например:

На iPhone X при 20 NPC FPS падает с 60 до 27 при въезде в городскую зону.

Для анализа можно использовать Unity Profiler.

30. Memory Testing

Необходимо контролировать:

- рост памяти;
- освобождение ресурсов;
- загрузку сцен;
- уничтожение объектов;
- Texture/Audio memory.

Полезный сценарий:

Scene A

→ Scene B

→ Scene A

→ Scene B

→ ...

Если память постоянно растет — возможна утечка или некорректное освобождение ресурсов.

Для анализа используется Unity Memory Profiler.

31. Unity Lifecycle Testing

Для мобильной игры обязательно проверять:

Game (игра)

↓

Background (свернуть)

↓

Foreground (снова открыть)

А также:

- блокировку экрана;
- входящий звонок;
- уведомления;
- сворачивание;

- выгрузку приложения ОС;
- восстановление.

Особенно внимательно проверять:

- сохранение;
- таймеры;
- рекламу;
- аудио;
- сетевое состояние;
- состояние гонки.

32. Physics / Collision Testing

Для Unity необходимо проверять:

- OnTriggerEnter;
- OnTriggerExit;
- OnCollisionEnter;
- Layers;
- Layer Collision Matrix;
- Rigidbody;
- Collider.

Bug Report желательно дополнить:

- Object
Layer
Collider
Position
Velocity
Scene
Build

Если столкновение происходит только иногда, обязательно указывать частоту воспроизведения.

33. Flaky Bug

Flaky Bug — дефект, который воспроизводится нестабильно.

Например:

3 раза из 10.

В Bug Report следует указывать:

Reproducibility: 30%

или:

Reproduced 3/10 times.

Для разработчика это может быть признаком:

Race Condition;

- timing issue;
- async/await проблемы;
- зависимости от FPS;
- проблемы с сетью;
- неправильного порядка событий.

Важно: нестабильный баг **не перестает быть багом** только потому, что воспроизводится не каждый раз.

34. Environment

Test Environment — окружение, в котором выполняется тест.

Для .NET:

- OS;
- .NET version;
- API version;
- Database version;
- configuration;
- environment variables.

Для Unity:

- Build version;
- Unity version;
- Platform;
- Device;
- OS version;
- GPU;
- RAM;
- Backend version.

35. Build Type

При Unity важно различать:

Development Build

Обычно:

- дополнительные логи;
- Profiler;
- debug information.

Release Build

Версия, максимально приближенная к той, которую получит пользователь.

Важно тестировать **оба типа**, поскольку поведение и производительность могут различаться.

36. Работа с логами

Для разработчика лог — один из важнейших источников информации.

При баге желательно собрать:

Steps

+

Video

+

Screenshot

+

Logs

+

Stack Trace

+

Environment

Для Unity:

- Player.log;
- Xcode logs;
- Android Logcat;
- Unity Profiler.

Для .NET:

- application logs;
- server logs;

- stack trace;
- HTTP request/response;
- database logs.

37. Network Tracing

При проблемах взаимодействия Unity ↔ .NET полезно посмотреть реальный HTTP-трафик.

Инструменты:

- Postman;
- Swagger;
- Charles Proxy;
- Fiddler;
- DevTools.

Необходимо проверить:

Request

↓

Headers

↓

Authentication

↓

Payload

↓

Status Code

↓

Response

Это позволяет определить, где находится проблема:

Unity

↓

Network

↓

.NET API

↓

Database

38. Жизненный цикл бага

Типовой workflow:

New



Assigned (назначенный)



In Progress



Fixed



Ready for Testing



Retest



Verified



Closed

Если проблема осталась:

Retest



Failed



Reopened

Дополнительные состояния:

Deferred

Баг отложен.

Причина:

Исправление не является приоритетным сейчас.

Can't Reproduce

Разработчик или QA не смог воспроизвести проблему.

Это не означает автоматически:

«Бага нет».

Нужно запросить:

- видео;
- логи;

- build version;
- environment;
- точные шаги;
- состояние данных;
- частоту воспроизведения.

39. Практическая методика тестирования новой функции

Для любой новой функции можно использовать следующий алгоритм.

Шаг 1. Изучить требования

Определить:

Что должна делать функция?

Что запрещено?

Какие ограничения существуют?

Шаг 2. Определить Expected Result

Нельзя начинать полноценное тестирование, не понимая ожидаемого результата.

Плохо:

Нажать Start.

Хорошо:

Нажать Start → загрузить гоночную сцену → создать игрока → активировать управление → начать таймер гонки.

Шаг 3. Проверить Happy/Positive Path

Работа при правильных данных.

Шаг 4. Проверить Negative Path

Что произойдет при:

- неправильных данных;
- отсутствии объекта;
- null;
- отсутствии сети;
- недостатке ресурсов.

Шаг 5. Проверить границы

Например:

0
1
MAX - 1
MAX
MAX + 1

Шаг 6. Проверить комбинации

Использовать:

- Decision Table;
- Pairwise Testing;
- State Transition Testing.

Шаг 7. Проверить неожиданные действия

Например:

- двойной клик;
- быстрое нажатие;
- Back;
- закрытие окна;
- смена сцены;
- отключение интернета.

Шаг 8. Проверить регрессию

Убедиться, что новая функциональность не повредила существующую.

40. Методика тестирования Unity-фичи

Например, добавлена механика смены передачи.

Functional (функционал)

Gear 1
→ Gear 2
→ Gear 3

Boundary (границы)

Gear 0
Gear 1
Gear 3
Gear 4

Input (ввод)

Проверяем:

- свайп вверх;

- свайп вниз;
- быстрые свайпы;
- несколько свайпов подряд.

State

Race Start

- Gear Change
- Corner
- Crash
- Finish

Performance

Проверяем влияние новой механики на FPS.

Regression

Проверяем:

- скорость;
- повороты;
- столкновения;
- UI;
- сохранение.

41. Методика тестирования .NET API

Например:

POST /api/cars/purchase

Проверяем:

Positive

valid user
valid car
enough coins

Negative

invalid user
invalid car
insufficient coins
negative price
null request
duplicate request
expired token

HTTP

Проверяем:

200
400
401
403
404
409
429
500

Data

Проверяем:

Database

↓

Balance

↓

Purchase

↓

Transaction

Concurrency

Два запроса одновременно:

Purchase

Purchase

Нельзя допустить двойное списание или двойную покупку.

42. Что должен проверять разработчик перед передачей задачи QA

Функциональность

- ☐ Happy Path работает
- ☐ Negative Path проверен
- ☐ Граничные значения проверены
- ☐ Неожиданные действия проверены

UI

- ☐ Все кнопки работают
- ☐ Нет обрезанного текста
- ☐ Нет наложения элементов

- ☐ Проверена локализация

Данные

- ☐ Save работает
- ☐ Load работает
- ☐ Некорректные данные обрабатываются

Errors

- ☐ Нет неожиданных Exceptions
- ☐ Нет Crash
- ☐ Логи не содержат критических ошибок

Integration

- ☐ API работает
- ☐ Database корректно изменяется
- ☐ Unity корректно обрабатывает ответ сервера

Regression

- ☐ Основные старые функции работают

43. Что особенно важно для Unity-разработчика

Перед релизом игры особое внимание уделить:

Gameplay

- ☐ управление;
- ☐ столкновения;
- ☐ камера;
- ☐ победа/поражение;
- ☐ прогресс.

Save System

- ☐ сохранение;
- ☐ загрузка;
- ☐ повторный запуск;
- ☐ поврежденные данные.

Mobile

- ☐ Background / Foreground;
- ☐ блокировка экрана;
- ☐ уведомления;
- ☐ разные разрешения;
- ☐ разные устройства.

Performance

- ☐ FPS;
- ☐ CPU;
- ☐ GPU;
- ☐ Memory;
- ☐ загрузка сцен.

Monetization

- ☐ Ads;
- ☐ rewarded ads;
- ☐ IAP;
- ☐ восстановление покупок.

44. Что особенно важно для .NET-разработчика

Особое внимание:

API

- ☐ HTTP Status Codes
- ☐ Validation
- ☐ Authentication
- ☐ Authorization
- ☐ Error Handling

Database

- ☐ CRUD
- ☐ Constraints
- ☐ Transactions
- ☐ Concurrent access

Security

- ☐ invalid token
- ☐ expired token
- ☐ unauthorized access
- ☐ input validation

Performance

- ☐ response time
- ☐ concurrent requests
- ☐ rate limiting

45. Ключевые вопросы разработчика при тестировании

Полезно превратить это в привычку.

Для каждой функции спросить:

1. Happy Path

Что будет, если пользователь делает все правильно?

2. Negative Path

Что будет, если пользователь делает неправильно?

3. Boundary

Что будет на границе допустимого значения?

4. State

Что будет при переходе из одного состояния в другое?

5. Concurrency

Что будет, если два действия произойдут одновременно?

6. Persistence

Что будет после перезапуска?

7. Environment

Что будет на другом устройстве, ОС или конфигурации?

8. Integration

Что будет, если соседняя система вернет ошибку?

9. Regression

Что я мог сломать своим изменением?

46. Минимальный набор терминов для разработчика

Если материал используется как учебная программа, можно разделить термины на два уровня.

Обязательный минимум

Разработчик должен уверенно понимать:

SUT - тестируемая система

Requirement – требования

Acceptance Criteria – допустимые критерии

Test Scenario – высокоуровневое описание теста

Test Case – конкретное описание теста (шаги, вводные, ожидаемые результаты)

Checklist – простой список задач для проверки (менее строгий TC)

Expected Result – ожидаемый результат

Actual Result – фактический результат

Bug / Error / Defect / Failed – общее понятие ошибки / логическая ошибка человека / следствие ошибки / проявление дефекта

Severity – важность (как сильно сломано) исправления ошибки

Priority – приоритет (скорость) исправления ошибки

Smoke Testing – быстрый тест

Sanity Testing – быстрая проверка чего то конкретного (например после изменений или добавлений)

Regression Testing – проверка старых функций после добавления новых

Retest – перепроверка после исправления

Functional Testing – проверка контрфактных функций

Positive Testing – проверка идеального сценария

Negative Testing – проверка неидеального сценария

Exploratory Testing – исследовательский тест (изучить и придумать как сломать)

Boundary Value Analysis – анализ граничных значений (наиболее потенциальные ошибки там)

Equivalence Partitioning – разделение на эквивалентные группы, чтобы не проверять каждый неверный символ/значение

State Transition Testing – проверка переходов состояний

Test Environment – проверка окружений (ОС, сервера, БД)

Test Data – проверка целостности данных

Bug Lifecycle – жизненный цикл ошибки (где, когда, периодичность)

Следующий уровень

После освоения основы:

Pairwise Testing – парный тест, помогает сократить все тесты, но не покрывает большую часть потенциальных ошибок

Decision Table – таблица соответствий (как система ведет себя на ошибку)

Integration Testing – проверка взаимодействия компонентов

System Testing – проверка системы целиком

E2E Testing – последовательная проверка как бы сделал пользователь от начала до конца

Acceptance Testing – проверка соответствия требованиям продукта

API Testing – проверка API

Idempotency – свойство, которое при повторном использовании не приводит к нежелательному результату

Rate Limiting – ограничения количества запросов

Concurrency Testing – тесты конкурентных запросов относительно потоков выполнения

Performance Testing – тесты производительности

Memory Testing – тесты потребляемой памяти

Compatibility Testing – тесты совместимости (API, окружения, БД, сервера-клиента)

Authentication – это не просто проверка кнопки «Войти». Это комплексный процесс, где нужно одновременно убедиться в корректности логики, безопасности и устойчивости к разным видам атак. Я подобрала ключевые моменты, на которые стоит обратить внимание.

Authorization – это проверка того, разрешён ли пользователю доступ к конкретному ресурсу или действию после успешной аутентификации.

Flaky Bug – ошибка появляющаяся не каждый раз

Black Box Testing – функциональное тестирование с точки зрения внешнего наблюдателя

White Box Testing – функциональное тестирование когда есть доступ к исходному коду

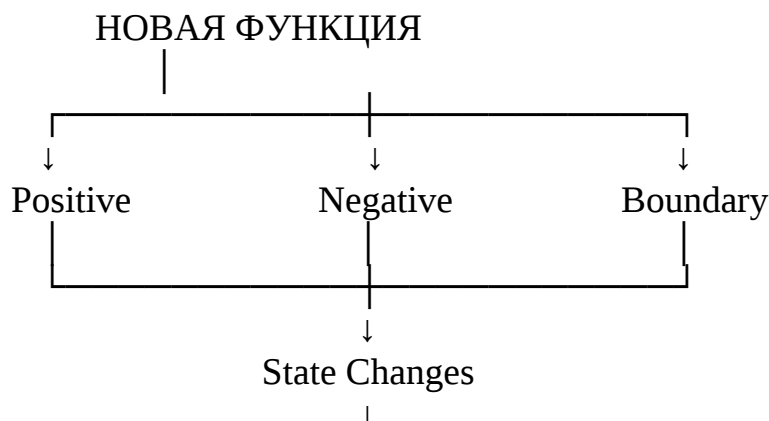
Gray Box Testing – гибридный подход **Black and White box**

47. Итоговая модель мышления

Главная идея методики:

Разработчик не должен просто проверять, работает ли его код. Он должен пытаться доказать, что код не работает в каком-либо сценарии.

При реализации новой функции можно использовать простую модель:



Combinations



Integration



Persistence



Environment



Regression

И самое важное практическое правило:

**Не ограничивайся проверкой сценария, который написал сам.
Проверь сценарии, которые мог бы придумать пользователь, QA-инженер и сама система.**

Для .NET-разработчика это означает особое внимание к **валидации, API, БД, concurrency и обработке ошибок.**

Для Unity-разработчика — к **состояниям, сохранениям, lifecycle, производительности, ресурсам, вводу и поведению на реальном устройстве.**

Такой подход превращает ручное тестирование из набора случайных кликов в **систематический инженерный процесс тест-дизайна и поиска дефектов.**